

Universidade de São Paulo
Faculdade de Filosofia, Ciências e Letras de Ribeirão Preto
Trabalho Integrado de Sistemas Distribuídos e Introdução ao
Desenvolvimento Web

Gustavo Bender
Guilherme Munhoz Serradilha

Relatório Técnico

Xadrez Multiplayer Cooperativo

Relatório técnico apresentado como parte do
Trabalho Integrado das disciplinas de Sistemas
Distribuídos e Introdução ao Desenvolvimento
Web.

Repositório do trabalho: <https://github.com/UmBender/xadrez>

Ribeirão Preto
2026

Sumário

Resumo	2
1 Introdução	2
2 Objetivos do Sistema	2
3 Arquitetura Geral	2
4 Tecnologias Utilizadas	3
5 Frontend	3
5.1 Arquivos Principais	4
5.2 Inicialização e Sessão	4
5.3 Autenticação	4
5.4 Menu Principal e Responsividade	4
5.5 Salas de Jogo	5
5.6 Partida em Tempo Real	5
5.7 Tabuleiro e Envio de Lances	5
5.8 Modos Colaborativos	6
5.9 Empate, Rendição e Revanche	6
5.10 Histórico e Replay	6
6 Backend	6
6.1 Organização em Camadas	6
6.2 Inicialização	7
6.3 Domínio	7
6.4 Autenticação e Tokens	7
6.5 Persistência	8
6.5.1 Coleção de Usuários	8
6.5.2 Coleção de Partidas	8
6.6 Rotas REST	9
6.7 WebSocket	9
6.8 Gerenciamento de Salas	10
6.9 Regras de Jogo	10
6.10 Concorrência	10
7 Integração Entre Frontend e Backend	10
8 Persistência de Dados	11
9 Testes e Verificação	11
10 Atendimento aos Requisitos do Trabalho	11
11 Pontos Fortes	12
12 Limitações e Melhorias Futuras	12
13 Conclusão	13

Resumo

Este relatório descreve a aplicação Xadrez Multiplayer Cooperativo, desenvolvida como trabalho integrado das disciplinas de Sistemas Distribuídos e Introdução ao Desenvolvimento Web. O sistema segue uma arquitetura multitier, com frontend em Flutter/Dart, backend em Go e persistência em MongoDB. A aplicação combina API RESTful para autenticação, salas e histórico com WebSocket para sincronização de partidas em tempo real. Além do modo clássico 1v1, o projeto implementa modos colaborativos 2v2 e 3v3, nos quais a interface e o servidor coordenam papéis, turnos, propostas de movimento, empate, rendição, revanche e histórico de partidas.

1 Introdução

O enunciado do trabalho integrado solicita a construção de uma aplicação web baseada em arquitetura multitier, com separação entre interface, processamento/logica de negócio e camada de dados. Também exige persistência, interface web interativa e responsiva, uso de API RESTful e possibilidade de substituição parcial por WebSocket quando houver necessidade de atualização em tempo real.

O projeto desenvolvido é uma aplicação de xadrez multiplayer com proposta cooperativa. O usuário pode criar conta, entrar no sistema, criar ou acessar salas, escolher equipe, jogar partidas em tempo real e consultar histórico. A aplicação também oferece replay de partidas salvas, permitindo revisar lances já persistidos.

O código-fonte do trabalho está disponível no repositório público: <https://github.com/UmBender/xadrez>.

2 Objetivos do Sistema

O objetivo principal do sistema é permitir partidas de xadrez online entre múltiplos usuários, explorando comunicação cliente-servidor, sincronização em tempo real e persistência de dados.

Os objetivos específicos são:

- oferecer cadastro e login de usuários;
- permitir criação e listagem de salas de jogo;
- sincronizar o estado do tabuleiro entre jogadores em tempo real;
- suportar modos 1v1, 2v2 e 3v3;
- registrar histórico de partidas e movimentos;
- permitir replay das partidas finalizadas;
- demonstrar separação entre frontend, backend e banco de dados.

3 Arquitetura Geral

A arquitetura do projeto é organizada em camadas. O frontend representa a camada de apresentação, responsável por renderizar telas, capturar interações e consumir os serviços do backend. O backend representa a camada de processamento e regras

de negocio, validando usuários, gerenciando salas, aplicando regras de jogo e coordenando mensagens WebSocket. O MongoDB representa a camada de dados, armazenando usuários e partidas.

Essa divisão atende ao requisito de arquitetura multitier, pois as responsabilidades ficam separadas:

- **Apresentação:** Flutter/Dart, no diretório `front/`;
- **Processamento:** Go, no diretório `back/`;
- **Dados:** MongoDB, acessado pelos repositórios do backend;
- **Comunicação:** REST para operações gerais e WebSocket para partidas.

4 Tecnologias Utilizadas

No frontend, foram identificadas as seguintes tecnologias e bibliotecas:

- Flutter Material, para construção da interface;
- Dart, como linguagem da camada de apresentação;
- pacote `http`, para requisições REST;
- `shared_preferences`, para manter o usuário salvo localmente;
- `web_socket_channel`, para comunicação em tempo real;
- pacote `chess`, para reconstruir partidas no replay.

No backend, foram utilizadas:

- Go 1.25;
- MongoDB, com o driver oficial `mongo-driver`;
- `gorilla/websocket`, para conexões WebSocket;
- `bcrypt`, para hash de senhas;
- `github.com/corentings/chess`, para validação e manipulação de partidas de xadrez.

5 Frontend

O frontend esta no diretório `front/` e foi estruturado como um conjunto de telas Flutter. Ele é responsável por apresentar a aplicação ao usuário e traduzir suas ações em chamadas REST ou mensagens WebSocket.

5.1 Arquivos Principais

Os arquivos principais do frontend são:

- `front/main.dart`: inicialização, login, cadastro, menu principal e histórico;
- `front/room.dart`: listagem e criação de salas;
- `front/gamescreen.dart`: tela da partida, tabuleiro e WebSocket;
- `front/match_viewer.dart`: replay de partidas salvas;

5.2 Inicialização e Sessão

A aplicação inicia em `main.dart`. Antes de renderizar a interface, o código inicializa o Flutter e consulta o `SharedPreferences` para verificar se existe um nome de usuário salvo. Se houver usuário persistido localmente, o sistema abre o menu principal. Caso contrário, apresenta a tela de autenticação.

Esse fluxo simplifica a experiência do usuário, pois evita novo login a cada abertura da aplicação. O logout remove o nome salvo e retorna para a tela de login.

5.3 Autenticação

A tela de autenticação alterna entre cadastro e login. No cadastro, o frontend envia uma requisição POST para:

- <https://xadrez-a8qm.onrender.com/api/register>

No login, envia uma requisição POST para:

- <https://xadrez-a8qm.onrender.com/api/login>

O corpo das requisições contém `username` e `password`. Quando o login é bem-sucedido, o nome de usuário é salvo localmente e a aplicação redireciona para o menu principal.

5.4 Menu Principal e Responsividade

O menu principal permite escolher o modo de jogo e acessar as principais funcionalidades. Os modos disponíveis são:

- 1v1: xadrez clássico;
- 2v2: modo em duplas;
- 3v3: modo conselho.

As ações disponíveis são criar partida, listar salas abertas e consultar histórico. O menu usa `MediaQuery` para adaptar o layout ao tamanho da tela. Em telas largas, os modos e as ações aparecem lado a lado. Em telas menores, os elementos são reorganizados em coluna.

5.5 Salas de Jogo

A tela `room.dart` consulta o endpoint:

- <https://xadrez-a8qm.onrender.com/api/rooms>

O retorno contém as salas disponíveis, com identificador, nome, número de jogadores e modo de jogo. O usuário também pode criar uma nova sala; nesse caso, o frontend gera um código aleatório de quatro caracteres e solicita a escolha da equipe, representada por `w` para brancas ou `b` para pretas.

5.6 Partida em Tempo Real

A tela de partida, em `gamescreen.dart`, abre uma conexão WebSocket com o backend:

- <wss://xadrez-a8qm.onrender.com/ws/play>

Os parâmetros enviados na URL identificam sala, usuário, modo e equipe. Por essa conexão, o frontend recebe o estado da partida e envia as ações do jogador.

O estado recebido pelo WebSocket inclui:

- FEN atual do tabuleiro;
- status da partida;
- lances válidos;
- quantidade de jogadores;
- jogadores conectados;
- papéis ativos;
- propostas de movimento;
- votos de revanche;
- oferta de empate.

5.7 Tabuleiro e Envio de Lances

O tabuleiro é reconstruído a partir do FEN enviado pelo backend. A interface interpreta letras como peças, números como casas vazias e barras como troca de linha. As peças são exibidas por imagens remotas e as casas recebem marcações visuais para seleção, destinos válidos e lances em disputa.

O envio de lances segue o formato UCI. Um lance comum pode ser representado por `e2e4`; uma promoção pode ser representada por `e7e8q`. Antes do envio, a interface consulta a lista de lances válidos recebida do backend. Se o lance envolver promoção de peão, um diálogo permite escolher rainha, torre, bispo ou cavalo.

5.8 Modos Colaborativos

No modo 1v1, a interface exibe um jogador por lado. No modo 2v2, exibe dois jogadores por equipe e respeita os papéis ativos definidos pelo backend. No modo 3v3, a interface suporta a regra de conselho: dois jogadores propõem lances e, se houver divergência, um terceiro jogador decide qual lance será executado.

Esse comportamento torna o sistema mais rico do que uma partida simples de xadrez online, pois exige coordenação entre vários usuários conectados simultaneamente.

5.9 Empate, Rendição e Revanche

Durante a partida, o usuário pode pedir empate, aceitar empate, render-se ou solicitar revanche após o fim do jogo. Essas ações são enviadas pelo WebSocket como mensagens especiais:

- `offer_draw`;
- `resign`;
- `rematch`.

Quando a partida termina, a interface mostra um diálogo com o resultado, indicando vitória das brancas, vitória das pretas ou empate.

5.10 Histórico e Replay

O histórico consulta o endpoint:

- <https://xadrez-a8qm.onrender.com/api/history>

A tela apresenta modo, equipes, data e resultado relativo ao usuário logado. Ao selecionar uma partida com lances salvos, o usuário abre a tela `match_viewer.dart`, que usa a biblioteca `chess` para reconstruir a partida lance a lance.

6 Backend

O backend está no diretório `back/` e foi implementado em Go. Ele concentra a autenticação, a lógica das salas, a validação das jogadas, a sincronização WebSocket e a persistência em MongoDB.

6.1 Organização em Camadas

O backend foi dividido em pacotes com responsabilidades claras:

- `cmd/api`: ponto de entrada da aplicação;
- `internal/delivery`: rotas HTTP, CORS, WebSocket e serialização JSON;
- `internal/service`: regras de negócio, autenticação, tokens e salas;
- `internal/repository`: acesso ao MongoDB;

- `internal/domain`: modelos e interfaces;
- `pkg/config`: carregamento de configuração por variáveis de ambiente.

Essa organização favorece manutenção, testes e separação entre entrada, negocio e dados.

6.2 Inicialização

A inicialização ocorre em `back/cmd/api/main.go`. O servidor carrega configurações, conecta ao MongoDB, cria repositórios, instancia serviços, registra rotas e inicia o servidor HTTP.

As variáveis de ambiente principais são:

- `MONGO_URI`;
- `JWT_SECRET`;
- `PORT`;
- `MONGO_DATABASE`;
- `MONGO_USERS_COLLECTION`;
- `MONGO_MATCHES_COLLECTION`;
- `JWT_TTL_MINUTES`.

6.3 Domínio

O pacote `internal/domain` define as estruturas centrais do sistema. O usuário possui nome, senha e salt. A partida registra identificador, modo, FEN atual, jogadores por papel, status, data e lista de movimentos. As mensagens WebSocket representam tanto movimentos enviados pelo cliente quanto respostas de estado enviadas pelo servidor.

As interfaces `UserRepository` e `MatchRepository` isolam a camada de negocio da implementação concreta de persistência.

6.4 Autenticação e Tokens

O serviço de autenticação valida dados de entrada, verifica usuários existentes e salva novos usuários no MongoDB. A senha não é armazenada em texto puro. O backend gera um salt aleatório, concatena esse salt com a senha e aplica `bcrypt`.

No login, o backend busca o usuário, compara o hash da senha e gera um token assinado com HMAC SHA-256. O token possui sujeito, instante de emissão e instante de expiração.

6.5 Persistência

A persistência é feita em MongoDB, um banco de dados não relacional orientado a documentos. Essa escolha combina bem com a aplicação porque os registros do sistema possuem estruturas naturalmente representáveis em JSON/BSON, como usuários, partidas, listas de movimentos e campos opcionais para os diferentes modos de jogo.

O backend não acessa o banco diretamente a partir das rotas. O acesso é concentrado no pacote `internal/repository`, que implementa interfaces definidas em `internal/domain`. Essa separação permite que a camada de serviços trabalhe com contratos, como `UserRepository` e `MatchRepository`, sem depender dos detalhes internos do MongoDB.

A conexão com o banco é configurada na inicialização do servidor. O arquivo `back/cmd/api/main.go` carrega as variáveis de ambiente pelo pacote `pkg/config`, cria o cliente MongoDB e seleciona o banco e as coleções. As principais configurações relacionadas ao banco são:

- `MONGO_URI`: endereço de conexão com o MongoDB;
- `MONGO_DATABASE`: nome do banco utilizado pela aplicação;
- `MONGO_USERS_COLLECTION`: nome da coleção de usuários;
- `MONGO_MATCHES_COLLECTION`: nome da coleção de partidas.

Quando essas variáveis opcionais não são informadas, o backend usa valores padrão definidos em `config.go`: banco `auth_db`, coleção `users` para usuários e coleção `matches` para partidas. A variável `MONGO_URI` é obrigatória, pois sem ela o servidor não consegue estabelecer conexão com o banco.

6.5.1 Coleção de Usuários

A coleção de usuários armazena as credenciais necessárias para autenticação. Cada documento possui:

- `username`: nome do usuário;
- `password`: senha processada com `bcrypt`;
- `salt`: valor aleatório usado junto com a senha antes do hash.

No cadastro, o serviço de autenticação verifica se o nome de usuário já existe. Se não existir, gera um salt, cria o hash da senha e chama o repositório de usuários para inserir o novo documento. No login, o repositório busca o documento pelo campo `username`; em seguida, o serviço compara a senha informada com o hash armazenado.

6.5.2 Coleção de Partidas

A coleção de partidas registra o histórico do jogo. Cada documento de partida armazena:

- `_id`: identificador da sala ou partida;

- **mode**: modo de jogo, como 1v1, 2v2 ou 3v3;
- **current_fen**: estado atual do tabuleiro em formato FEN;
- nomes dos jogadores nos papéis **white_name**, **black_name**, **w2_name**, **b2_name**, **w3_name** e **b3_name**;
- **status**: resultado ou estado da partida;
- **date**: data registrada;
- **moves**: lista de movimentos em formato UCI.

O repositório de partidas salva os dados com **upsert**. Isso significa que, se a partida ainda não existe na coleção, ela é criada; se já existe, seus campos são atualizados. Esse comportamento é útil porque a mesma sala pode receber novas atualizações de estado ao longo do jogo, sem exigir que o backend trate manualmente a diferença entre criação e atualização.

A busca de histórico considera todos os papéis possíveis ocupados por um jogador: **white_name**, **black_name**, **w2_name**, **b2_name**, **w3_name** e **b3_name**. Dessa forma, o histórico funciona para partidas 1v1, 2v2 e 3v3.

Essa estrutura também permite que o frontend recupere a lista de partidas de um usuário e abra o replay. O replay depende especialmente do campo **moves**, pois a tela **match_viewer.dart** reconstrói o tabuleiro aplicando os movimentos em ordem.

6.6 Rotas REST

As rotas REST registradas são:

- **POST /api/register**: cadastro de usuário;
- **POST /api/login**: login;
- **GET /api/rooms**: listagem de salas disponíveis;
- **GET /api/history**: consulta de histórico por usuário.

O servidor também configura CORS, permitindo a comunicação entre frontend e backend mesmo quando executados em origens diferentes.

6.7 WebSocket

O endpoint WebSocket é:

- **WS /ws/play**

Quando um cliente se conecta, o backend identifica sala, usuário, modo e equipe. Em seguida, registra o cliente na sala, atribui um papel e envia o estado atualizado para todos os participantes. Durante a partida, cada mensagem recebida é processada como movimento, pedido de empate, rendição ou revanche.

6.8 Gerenciamento de Salas

O `RoomManager` mantém salas em memória e controla criação, entrada, saída, listagem e exclusão de salas vazias. Cada sala possui estado próprio, incluindo jogo, movimentos, clientes, propostas de movimento, votos de revanche e oferta de empate.

O número máximo de jogadores depende do modo:

- 1v1: 2 jogadores;
- 2v2: 4 jogadores;
- 3v3: 6 jogadores.

6.9 Regras de Jogo

No modo 1v1, os papéis ativos alternam entre `w1` e `b1`. No modo 2v2, a alternância segue a ordem `w1`, `b1`, `w2`, `b2`. No modo 3v3, dois jogadores da equipe propõem lances e um terceiro atua como decisor quando há divergência.

O backend usa a biblioteca de xadrez para validar e executar lances. Após um lance válido, o servidor atualiza a lista de movimentos, limpa propostas pendentes, atualiza o FEN e persiste o estado da partida.

6.10 Concorrência

Como o sistema trabalha com múltiplas conexões simultâneas, o backend usa mecanismos de sincronização:

- `sync.RWMutex` para proteger salas;
- `sync.RWMutex` para proteger o mapa de clientes conectados;
- `atomic` para gerar identificadores de clientes;
- locks curtos durante alterações do estado compartilhado.

Esse cuidado é essencial em um sistema distribuído com vários usuários enviando mensagens em tempo real.

7 Integração Entre Frontend e Backend

A integração entre as camadas ocorre por REST e WebSocket. O REST é usado para operações pontuais, como cadastro, login, consulta de salas e histórico. O WebSocket é usado para a partida, pois o estado do tabuleiro precisa ser atualizado imediatamente para todos os jogadores.

O fluxo resumido da aplicação é:

1. o usuário realiza cadastro ou login pelo frontend;
2. o frontend consulta ou cria uma sala;
3. o usuário escolhe a equipe;
4. o frontend abre WebSocket com sala, usuário, modo e equipe;

5. o backend registra o cliente e envia o estado da sala;
6. os jogadores enviam movimentos ou acoes especiais;
7. o backend valida, atualiza e persiste o estado;
8. o frontend recebe broadcasts e atualiza a interface.

8 Persistência de Dados

A persistência atende duas necessidades principais. A primeira é armazenar usuários com senha protegida por salt e hash. A segunda é armazenar partidas, incluindo FEN, jogadores, status, data e lista de movimentos.

O histórico de partidas no frontend depende diretamente desses registros. O replay também se apoia na lista de movimentos persistida, mostrando como a camada de dados contribui para funcionalidades visíveis ao usuário.

9 Testes e Verificação

O backend possui testes automatizados para autenticação, tokens, salas, rotas HTTP, WebSocket e configuracao. Durante a verificação local, o comando `go test ./...` foi executado com sucesso no diretório `back/`.

Os testes cobrem:

- hash e validação de senha;
- geração e validação de token;
- entrada em sala e controle de capacidade;
- alternância de papeis;
- decisão no modo 3v3;
- persistência de lances;
- rotas REST;
- fluxo WebSocket.

10 Atendimento aos Requisitos do Trabalho

O sistema atende aos requisitos técnicos do enunciado da seguinte forma:

- **Arquitetura multitier:** frontend, backend e banco de dados estão separados por responsabilidades;
- **Persistência de dados:** usuários e partidas são armazenados em MongoDB;
- **Interface interativa e responsiva:** o frontend apresenta telas interativas, menu adaptável e tabuleiro atualizado em tempo real;
- **API RESTful:** cadastro, login, salas e histórico usam endpoints REST;

- **WebSocket:** partidas são sincronizadas por WebSocket, substituindo parte da comunicação REST onde o tempo real é necessário;
- **Relatório técnico:** este documento descreve arquitetura, decisões de projeto, tecnologias e funcionamento das camadas.

11 Pontos Fortes

Entre os pontos fortes do projeto, destacam-se:

- separação clara entre frontend, backend e persistência;
- uso adequado de WebSocket para comunicação em tempo real;
- suporte a modos colaborativos além do xadrez clássico;
- histórico e replay de partidas;
- senhas protegidas por salt e `bcrypt`;
- backend com testes automatizados;
- regras de negocio concentradas em serviços no backend;
- interface com feedback visual, diálogos e indicação de jogadores ativos.

12 Limitações e Melhorias Futuras

Algumas limitações foram identificadas durante a análise:

- as URLs do backend estão fixas no código do frontend;
- o frontend salva localmente apenas o nome do usuário, embora o backend retorne token;
- as imagens das peças dependem de URLs externas;
- as chamadas HTTP ficam diretamente nas telas, sem uma camada própria de serviço no frontend;
- o estado das salas em andamento fica em memória no backend, embora o histórico das partidas seja persistido.

Como melhorias futuras, poderiam ser implementadas configurações por ambiente, uso efetivo do token no frontend, centralização das chamadas HTTP, empacotamento completo do projeto Flutter e persistência mais ampla do estado das salas.

13 Conclusão

O projeto Xadrez Multiplayer Cooperativo cumpre os principais requisitos do trabalho integrado. A aplicação combina interface Flutter, backend Go e MongoDB em uma arquitetura multitier clara. A comunicação REST atende opera coes convencionais, enquanto o WebSocket resolve a necessidade de sincronização em tempo real durante as partidas.

O sistema também apresenta uma proposta funcional diferenciada ao suportar modos 2v2 e 3v3, adicionando regras colaborativas ao xadrez online. O backend demonstra organização em camadas, controle de concorrência, persistência e testes. O frontend transforma esses recursos em uma experiencia interativa com login, salas, tabuleiro, histórico e replay.

Assim, a solução desenvolvida demonstra a integração entre conceitos de sistemas distribuídos e desenvolvimento web, atendendo aos critérios técnicos estabelecidos no enunciado e fornecendo uma base consistente para apresentação e avaliação do trabalho.